OFFSIDE LABS

Superstate Allowlist

Smart Contract Security Assessment

May 2025

Prepared for:

Superstate

Prepared by:

Offside Labs

Sirius Xie

Gongyu Shi

Contents

1 About Offside Labs

Offside Labs stands as a pre-eminent security research team, comprising highly skilled hackers with top - tier talent from both academia and industry.

The team demonstrates extensive and diverse expertise in modern software systems, which encompasses yet are not restricted to *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. Offside Labs is at the forefront of innovative domains such as *cryptocurrencies* and *blockchain technologies*. The team achieved notable accomplishments including the successful execution of remote jailbreaks on devices like the **iPhone** and **PlayStation 4**, as well as the identification and resolution of critical vulnerabilities within the **Tron Network**.

Offside Labs actively involves in and keeps contributing to the security community. The team was the winner and co-organizer for the *DEFCON CTF*, the most renowned CTF competition in Web2. The team also triumphed in the **Paradigm CTF 2023** in Web3. Meanwhile, the team has been conducting responsible disclosure of numerous vulnerabilities to leading technology companies, including *Apple*, *Google*, and *Microsoft*, safeguarding digital assets with an estimated value exceeding **$300 million**.

During the transition to Web3, Offside Labs has attained remarkable success. The team has earned over **$9 million** in bug bounties, and **three** of its innovative techniques were acknowledged as being among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

2 Executive Summary

Introduction

Offside Labs completed a security audit of *Superstate Allowlist* smart contracts, starting on April 24, 2025, and concluding on May 8, 2025.

Project Overview

The *Superstate Allowlist Program* is designed to manage the state of public equity. It can freeze, thaw, or burn the public equity tokens of specific users. For access control during the thaw operation, allowlists are maintained for both public and private instruments, as well as their associated users.

Audit Scope

The assessment scope contains mainly the smart contracts of the allowlist program for the *Superstate Allowlist* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Superstate Allowlist
  - Codebase: `https://github.com/superstateinc/solana-allowlist-program-audit`
  - Commit Hash: cab1688223db9f988c9038c411111dabac7089a2

We listed the files we have audited below:

- Superstate Allowlist
  - program/src/**/*.rs

Findings

The security audit revealed:

- 0 critical issue
- 1 high issues
- 2 medium issues
- 3 low issues
- 6 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Potential Bypass in Thaw IX Due to Missing Account Checks	High	Fixed
02	Missing Account Allocation May Cause PDA Initialization Failure	Medium	Fixed
03	Excessive Rent Transferred When Creating PDA Accounts	Medium	Fixed
04	Improper Rent Refund Logic in AllowedAccount Removal IXs	Low	Acknowledged
05	DoS Risk in InitializeAdminAuthority Account Creation	Low	Acknowledged
06	Potential Frontrunning Risk in Permissionless InitializeAdminAuthority IX	Low	Acknowledged
07	Redundant Signer Seeds When Invoking System Transfer IX	Informational	Fixed
08	Redundant Account When Invoking System Assign IX	Informational	Fixed
09	Redundant Or Unused Account in AdminFreeze IX	Informational	Fixed
10	Missing Validation on PrivateAllowedAccount.mint	Informational	Fixed
11	Missing Validation on ATA Account	Informational	Fixed
12	Missing Account Type May Lead to Type Confusion	Informational	Acknowledged

4 Key Findings and Recommendations

4.1 Potential Bypass in Thaw IX Due to Missing Account Checks

Severity: High	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

In the `Thaw` IX, the provided `user_private_allowed_account_pda` is expected to be a `PrivateAllowedAccount`.

```
334         if *is_private_instrument_mint {
335             // check if user is allowed for the private instrument associated
                ↪ with the mint
336             if let Ok(private_allowed_account) =
                ↪ borsh::from_slice::<PrivateAllowedAccount>(
337                 &user_private_allowed_account_pda.data.borrow_mut(),
338             ) {
339                 if private_allowed_account.owner ==
                    ↪ user_associated_token_account.base.owner {
340                     user_is_allowed = true;
341                 }
342             }
```

program/src/allowlist/processor.rs#L334-L342

However, several critical checks are missing:

- The account's `owner` is not verified to be the Allowlist program.
- It is not verified whether the account is indeed a valid PDA of the Allowlist program.
- The `private_allowed_account.mint` field is not checked against `mint_info`.
- The `private_allowed_account.mint` field is not checked against `private_allowlist.mint`.

Impact

An attacker could craft a malicious `private_allowed_account` with `user_is_allowed` set to `true`, bypass the validation, and use it to successfully thaw the `user_associated_token_account_info`.

A similar issue also exists with the `user_public_allowed_account_pda`, which is expected to be of type `PublicAllowedAccount`.

Recommendation

It is recommended to add the following validations for `user_private_allowed_account_pda` :

- Check that the account's `owner` matches the Allowlist program ID.
- Derive and verify the expected PDA using `user_account_info` and `mint_info` , and confirm it matches the passed-in account.
- Verify that `private_allowed_account.mint == mint_info` .
- Verify that `private_allowed_account.mint == private_allowlist.mint` .

And add the following validations for `user_public_allowed_account_pda` :

- Check that the account's `owner` matches the Allowlist program ID.
- Derive and verify the expected PDA using `user_account_info` , and confirm it matches the passed-in account.

Mitigation Review Log

Fixed in the commit 35496aa58f9d646046f9620bc72865d4a74978cb.

4.2 Missing Account Allocation May Cause PDA Initialization Failure

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In `create_pda_account` , if `new_pda_account` holds more than 0 lamports, the function transfers the required lamports for the specified space and updates the account's owner using `system_instruction::assign` .

```
        invoke_signed(
            &system_instruction::transfer(payer.key, new_pda_account.key,
↪ lamports_required),
            ...
        )?;
        invoke_signed(
            &system_instruction::assign(new_pda_account.key, owner),
            ...
        )?;
```

program/src/allowlist/tools/account.rs#L49-L66

However, this flow may be incomplete: if `new_pda_account.data_len()` is smaller than the required `space` , the account's data size does not meet the expected PDA size.

Impact

In such cases, the IX may never successfully create the `new_pda_account`, and subsequent operations may trigger an I/O error due to the insufficient account size, causing the IX to fail.

This issue affects multiple IXs, including `AdminAddPrivateAllowlist`, `AdminAddPublicAllowedAccount`, and `AdminAddPrivateAllowedAccount`.

Recommendation

It is recommended to add a `system_instruction::allocate` call before `system_instruction::assign` to ensure that `new_pda_account.data_len()` matches the `space` argument.

Mitigation Review Log

Fixed in the commit 35496aa58f9d646046f9620bc72865d4a74978cb.

4.3 Excessive Rent Transferred When Creating PDA Accounts

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In `create_pda_account`, if `new_pda_account` already holds more than 0 lamports, the function transfers the full amount required for the account's space from the payer.

```
let lamports_required = rent.minimum_balance(space).max(1);
if new_pda_account.data_is_empty() && new_pda_account.lamports() == 0 {
        ...
} else {
    ...
        invoke_signed(
            &system_instruction::transfer(payer.key, new_pda_account.key,
↪ lamports_required),
            ...
        )?;
```

program/src/allowlist/tools/account.rs#L18-L57

However, when `new_pda_account.lamports() > 0`, it's sufficient to transfer only `lamports_required - new_pda_account.lamports()` to successfully create the PDA.

Impact

This can lead to the payer spending more rent than necessary when creating the PDA account.

This behavior affects multiple IXs, including `AdminAddPrivateAllowlist`, `AdminAddPublicAllowedAccount`, and `AdminAddPrivateAllowedAccount`.

Recommendation

It is recommended to recalculate the amount to transfer based on the current lamports held by `new_pda_account` before calling `system_instruction::transfer`.

Mitigation Review Log

Fixed in the commit 35496aa58f9d646046f9620bc72865d4a74978cb.

4.4 Improper Rent Refund Logic in AllowedAccount Removal IXs

Severity: Low	Status: Acknowledged
Target: Smart Contract	Category: Logic Error

Description

In the `AdminAddPublicAllowedAccount` IX, a user's `PublicAllowedAccount` PDA is created. The rent is paid by the user if `user_account_to_add` is a signer; otherwise, it's covered by the admin.

In the `AdminRemovePublicAllowedAccount` IX, the user's `PublicAllowedAccount` is closed, and all rent lamports are transferred to the admin account. However, this rent might have originally been paid by the user, not the admin.

A similar issue also exists in the `AdminRemovePrivateAllowedAccount` IX.

Impact

For users, the rent they pay when creating an `AllowedAccount` is not refunded to them upon removal, which may negatively impact the user experience.

Recommendation

It is recommended to return the rent to the party who initially funded the PDA creation.

4.5 DoS Risk in InitializeAdminAuthority Account Creation

Severity: Low	Status: Acknowledged
Target: Smart Contract	Category: Logic Error

Description

The `InitializeAdminAuthority` IX creates a PDA account called `global_admin_pda`, which stores the admin authority's public key. This account is created using `system_instruction::create_account`.

However, the IX does not account for the possibility that the PDA might already have some lamports. Since `create_account` will fail if the destination account already holds any lamports, directly calling it can cause the IX to revert.

Impact

An attacker could exploit this by transferring a small amount of lamports to the `global_admin_pda` account before the admin invokes `InitializeAdminAuthority` IX, causing the `create_account` IX to fail and preventing the account from ever being properly created.

Recommendation

It is recommended to use the fixed `create_pda_account` utility function to create `global_admin_pda`.

4.6 Potential Frontrunning Risk in Permissionless InitializeAdminAuthority IX

Severity: Low	Status: Acknowledged
Target: Smart Contract	Category: Data Validation

Description

The `InitializeAdminAuthority` IX is currently permissionless, meaning anyone can invoke it. As part of its execution, it creates a `global_admin_pda`, which is the only PDA in the program associated with admin functionality. The `admin_authority` stored in this PDA is set to the `initial_admin_authority` provided in the IX.

If someone invokes this IX before the legitimate admin does, the PDA will end up recording the wrong admin address, breaking the expected admin setup for the allowlist.

Impact

An attacker could observe the program deployment and frontrun the real admin by invoking `InitializeAdminAuthority` IX first. This would effectively give the attacker control over the program's admin privileges, rendering the program unusable by the actual project team. In such a case, the only option may be to redeploy the program under a new program ID.

Recommendation

It is recommended to make this IX permissioned—e.g., by restricting invocation to a predefined authority account. Alternatively, project team have to ensure that the program deployment IX and `InitializeAdminAuthority` can be executed together in a single transaction to prevent the possibility of being frontrun.

4.7 Informational and Undetermined Issues

Redundant Signer Seeds When Invoking System Transfer IX

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In `create_pda_account`, when invoking `system_instruction::transfer`, the `signer_seeds` for `new_pda_account` are also passed into `invoke_signed`. However, in a system transfer IX, only the `from` account is required to be a signer. Therefore, it's unnecessary to pass in `new_pda_signer_seeds` when calling `invoke_signed` in this context.

Redundant Account When Invoking System Assign IX

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In `create_pda_account`, when invoking `system_instruction::assign`, the `payer` account is also included in the `account_infos` passed to `invoke_signed`. However, the system assign IX only requires the `new_pda_account` in its `account_infos`. It's recommended to remove the `payer` from the `account_infos` passed to `invoke_signed`.

Redundant Or Unused Account in AdminFreeze IX

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In the `AdminFreeze` IX, there is an account called `user_account_info`, but it is only used at the end of the IX to print a log message via `msg!`. The IX does not verify the relationship

between `user_account_info` and `user_associated_token_account_info`, so there's no guarantee that the ATA and the owner recorded in the log are actually related.

It is recommended to either add a check to ensure `user_associated_token_account.base.owner == user_account_info`, or remove the `user_account_info` reference from the log message.

Missing Validation on PrivateAllowedAccount.mint

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Data Validation

In the `AdminRemovePrivateAllowedAccount` IX, it requires that `allowed_account_to_remove.owner` must match `user_account_to_remove`. However, it does not check the relationship between `allowed_account_to_remove.mint` and `mint_info`. It is recommended to add this check to ensure the integrity of validation for `PrivateAllowedAccount` accounts.

Missing Validation on ATA Account

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Data Validation

In the `Thaw`, `AdminFreeze`, and `AdminBurn` IXs, the `user_associated_token_account_info` is passed in. Based on the variable name, it is expected to be an ATA (Associated Token Account). However, none of these IX implementations use functions like `get_associated_token_address_with_program_id` to verify whether the provided `user_associated_token_account_info` is actually an ATA.

Without this verification, even a regular token account that is not an ATA could still be used to successfully execute the IX.

Therefore, if the Allowlist design explicitly requires `user_associated_token_account_info` to be an ATA, it is recommended to validate it using `get_associated_token_address_with_program_id` to ensure it meets the requirement.

Missing Account Type May Lead to Type Confusion

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Data Validation

None of the accounts in the Allowlist have a dedicated field to indicate their `AccountType`, which can easily lead to type confusion or other security issues during future program development. For example, based solely on the account data, there's no way to distinguish between a `PublicAllowedAccount` and a `PrivateAllowlist` account.

It is recommended to add a `Type` field to these account structures, so that after deserialization, the program can explicitly verify the account's type and avoid potential type

confusion.

5 Disclaimer

This report reflects the security status of the project as of the date of the audit. It is intended solely for informational purposes and should not be used as investment advice. Despite carrying out a comprehensive review and analysis of the relevant smart contracts, it is important to note that Offside Labs' services do not encompass an exhaustive security assessment. The primary objective of the audit is to identify potential security vulnerabilities to the best of the team's ability; however, this audit does not guarantee that the project is entirely immune to future risks.

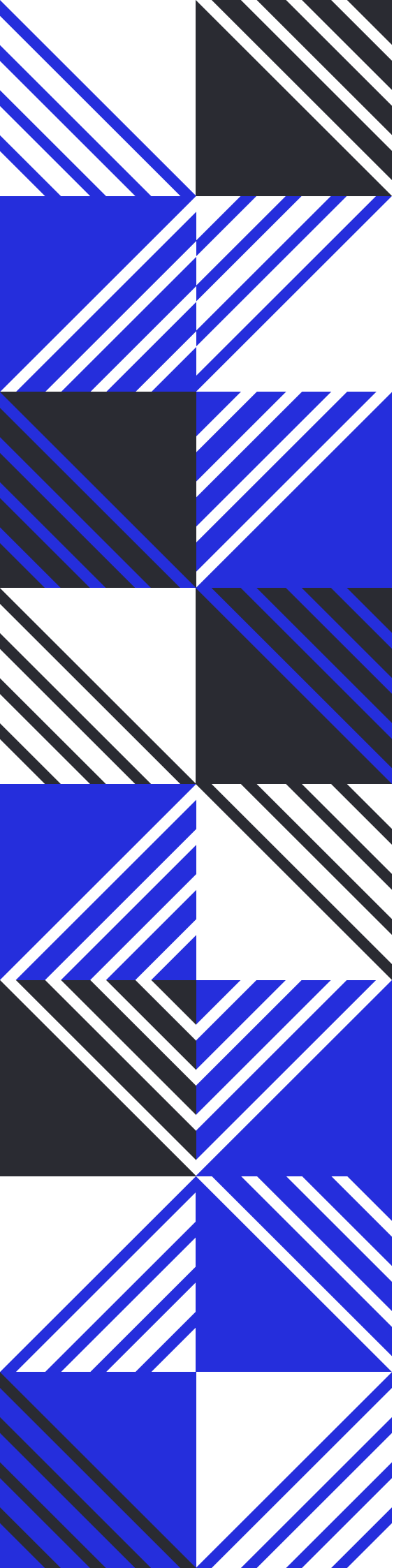
Offside Labs disclaims any liability for losses or damages resulting from the use of this report or from any future security breaches. The team strongly recommends that clients undertake multiple independent audits and implement a public bug bounty program to enhance the security of their smart contracts.

The audit is limited to the specific areas defined in Offside Labs' engagement and does not cover all potential risks or vulnerabilities. Security is an ongoing process, regular audits and monitoring are advised.

Please note: Offside Labs is not responsible for security issues stemming from developer errors or misconfigurations during contract deployment and does not assume liability for centralized governance risks within the project. The team is not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By utilizing this report, the client acknowledges the inherent limitations of the audit process and agrees that the firm shall not be held liable for any incidents that may occur after the completion of this audit.

This report should be considered null and void in case of any alteration.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs